



IdeaVault

A Deep Dive into the Codebase Architecture

MERN Stack — JavaScript

This document explains every file, every function, and every connection.

Version 1.0 · July 2026

Table of Contents

1. Introduction — What IdeaVault Is and How It Works
2. The Big Picture — Architecture at a Glance
3. How the Server Starts — From index.js to a Running API
4. Authentication — How Users Register, Login, and Stay Logged In
5. Ideas — The Core Domain Object
6. Voting — Binary Judgments with Change Capability
7. Comments — Discussion on Ideas
8. The Score Engine — How Votes Become a Percentage
9. The React Client — How the Frontend Is Structured
10. Page by Page — What Each React Component Does
11. Styling — How index.css Controls the Entire UI
12. The Connection — How Frontend Talks to Backend
13. Testing — How the Test Suite Works

1. Introduction — What IdeaVault Is and How It Works

IdeaVault is a web application built on the MERN stack — **MongoDB** for the database, **Express.js** for the backend server, **React** for the frontend user interface, and **Node.js** as the JavaScript runtime that ties everything together. The application's purpose is simple but powerful: users post ideas publicly, and the community votes them *good* or *bad*. Anyone can comment on any idea, and the original author has special powers — they can close their idea to prevent further voting, or delete comments they find inappropriate.

The entire application lives in a **monorepo** structure. This means the server code and the client code live side by side in the same Git repository, sharing a single root `package.json` that uses a tool called `concurrently` to start both the backend and frontend development servers simultaneously with a single command: `npm run dev`.

When you run this command, two things happen in parallel. The Express server starts on **port 5000**, listening for API requests. The Vite development server starts on **port 3000**, serving the React application to your browser. The Vite server is configured to *proxy* all requests to `/api/*` over to the Express server on port 5000. This means from the browser's perspective, everything appears to come from a single origin — port 3000 — even though the actual API work happens on a different port. This proxy trick avoids cross-origin issues during development.

The data model at the heart of IdeaVault consists of four MongoDB collections: **Users**, **Ideas**, **Votes**, and **Comments**. Users create Ideas. Other users cast binary Votes (good or bad) on those Ideas. Anyone can leave Comments on an Idea. The Idea's author has exclusive control over the Idea's status (open for voting, or closed) and can delete any Comment on their Idea. Votes are changeable — a user can switch their vote from good to bad, or remove it entirely. The system enforces one vote per user per idea through a compound unique index in MongoDB.

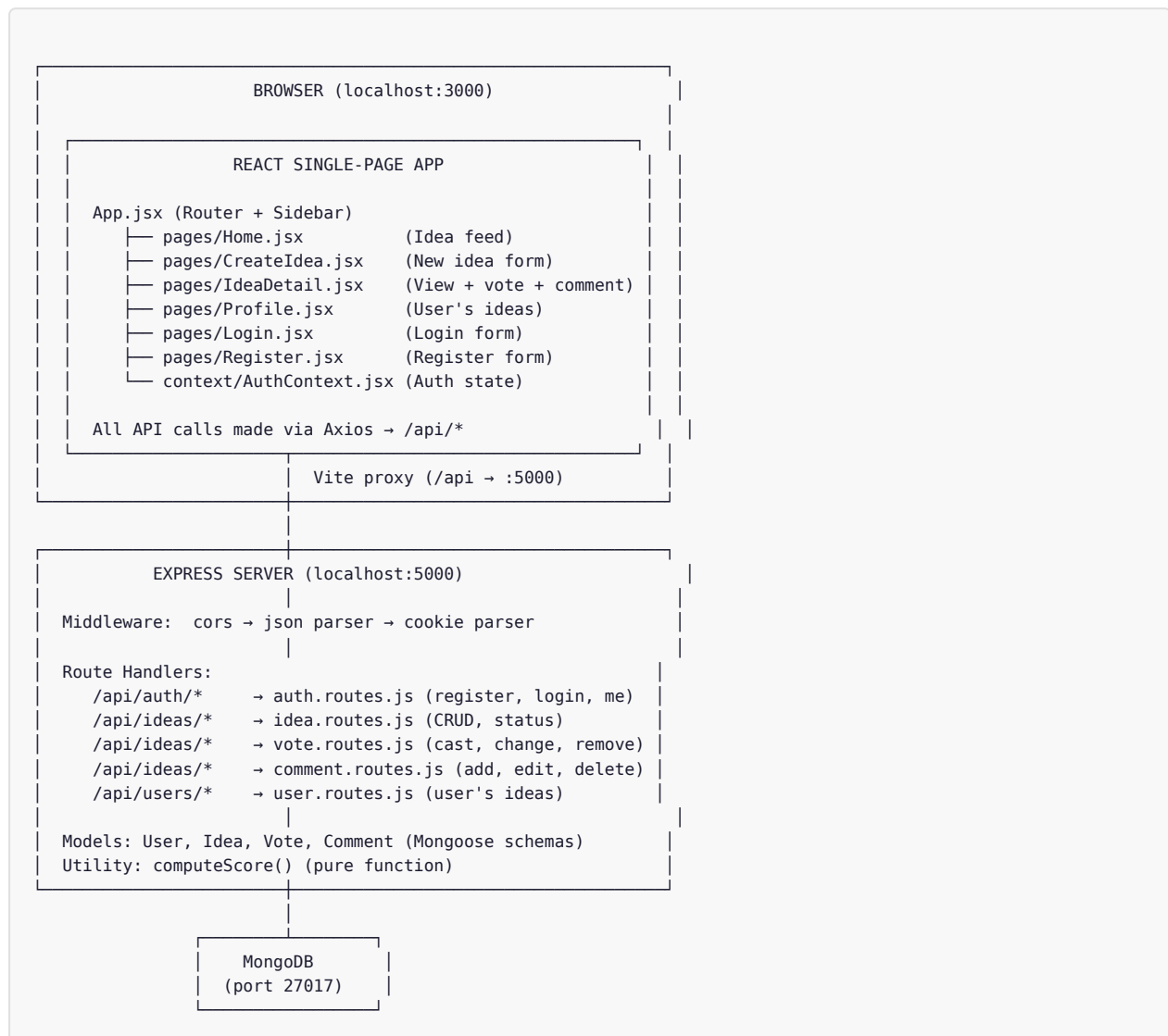
One important design decision: **the score is never stored in the database**. Instead, a pure function called `computeScore()` runs every time an Idea is read, looking at all the Votes for that Idea and calculating the good count, bad count, and percentage on the fly. This means scores are always up to date, but it does mean the database is queried for votes every time an Idea is fetched.

How to Read This Document

Each section of this document focuses on one aspect of the codebase. For every file, you will learn: what the file contains, what functions or components it defines, what other files it imports from, and what other files import from it. By the end, you should be able to open any file in the project and understand exactly what it does and how it connects to everything else.

2. The Big Picture — Architecture at a Glance

The application follows a classic three-tier architecture. At the top is the **presentation tier** — the React single-page application that runs in the user's browser. In the middle is the **logic tier** — the Express.js server that handles authentication, validates requests, enforces business rules, and exposes a RESTful API. At the bottom is the **data tier** — MongoDB, which stores users, ideas, votes, and comments as documents in collections.



Notice that three different route files — `idea.routes.js`, `vote.routes.js`, and `comment.routes.js` — are all mounted on the same path: `/api/ideas`. This is because votes and comments are *sub-resources* of ideas. An idea has votes, so the path is `/api/ideas/:id/votes`. An idea has comments, so the path is `/api/ideas/:id/comments`. Express allows multiple routers to be mounted on the same path, and each router only handles the routes it knows about.

File Count Summary

The project contains approximately 31 source files. The server has 15 source files (including 5 test files). The client has 9 source files (6 pages, 1 context, 1 root component, 1 entry point). Configuration and build files make up the rest.

3. How the Server Starts — From index.js to a Running API

Every journey through the IdeaVault codebase begins at `server/index.js`. This is the entry point — the file Node.js executes when you run `node index.js` or `npm run dev` in the server directory.

The Startup Sequence

When the server starts, three things happen in order:

First, the file calls `dotenv.config()`, which reads the `.env` file in the server directory and loads its key-value pairs into `process.env`. This is how the server learns the MongoDB connection URI (`MONGO_URI`), the JWT secret (`JWT_SECRET`), and the port number (`PORT`). Without this step, these values would be undefined.

Second, the file imports `./src/app.js`, which is the Express application. The import itself triggers all the setup code in `app.js` to execute: creating the Express app instance, attaching middleware (CORS, JSON parsing, cookie parsing), and mounting all the route handlers.

Third, the file connects to MongoDB using `mongoose.connect(process.env.MONGO_URI)`. Mongoose is an Object Document Mapper (ODM) that provides a structured way to interact with MongoDB. Once the connection is established, the server calls `app.listen(PORT)` to start accepting HTTP requests.

What Happens Inside app.js

The file `server/src/app.js` is where the Express application is assembled. It exports a single `app` object, which is the configured Express instance. Here is exactly what it does:

It imports five route modules and mounts them on specific URL paths. The auth routes handle `/api/auth/*`. The idea routes handle `/api/ideas/*`. The vote routes and comment routes also mount on `/api/ideas/*` because they handle sub-resources. The user routes handle `/api/users/*`. There is also a small inline handler for `/GET /api/health` that simply returns `{ status: "ok" }` to confirm the server is running.

Before any route is reached, every request passes through three middleware functions in order. First, **CORS middleware** adds headers to allow cross-origin requests from `http://localhost:3000` (the Vite dev server) with credentials. Second, **express.json()** parses any JSON body in the request. Third, **cookie-parser** parses the `Cookie` header into a JavaScript object on `req.cookies`.

At the bottom of `app.js`, there is an error-handling middleware. If any route handler throws an error or calls `next(error)`, this catch-all handler sends a JSON response with the error message and a 500 status code.

4. Authentication — How Users Register, Login, and Stay Logged In

Authentication in IdeaVault uses **JSON Web Tokens (JWT)** stored in **httpOnly cookies**. This is a security-conscious choice: because the token is in an httpOnly cookie, JavaScript running in the browser cannot access it, which means it is immune to cross-site scripting (XSS) attacks that try to steal the token.

The User Model

The User model (`server/src/auth/user.model.js`) defines four fields: `name` (a display name), `email` (unique, lowercased automatically), `password` (hashed, excluded from queries by default using `select: false`), and `createdAt` (automatically set).

The password field has `select: false` , which means that when you do `User.find()` or `User.findById()` , the password field is not returned. You must explicitly request it with `.select("+password")` . This is a safety measure: even if a developer forgets to exclude the password in a response, the model will not include it unless specifically asked.

The User model has a **pre-save hook** that runs before every new user is saved. If the password field has been modified (which it is when creating a new user or changing the password), it hashes the password using bcrypt with 10 salt rounds. This means the raw password is never stored in the database — only the hash.

The model also defines an instance method called `comparePassword` . Given a candidate password, it uses `bcrypt.compare` to check whether the candidate matches the stored hash. This method is called during login.

The Auth Service

The auth service (`server/src/auth/auth.service.js`) provides three helper functions that encapsulate JWT and cookie logic.

`generateToken(userId)` creates a JWT containing `{ id: userId }` , signed with the `JWT_SECRET` from environment variables, and set to expire in 7 days. This token is what proves a user's identity.

`setTokenCookie(res, token)` attaches the token to the HTTP response as an httpOnly cookie named "token". The cookie is set with `httpOnly: true` (inaccessible to JavaScript), `sameSite: "lax"` (allows same-site requests), and `maxAge: 7 days` . In production, the `secure` flag is also set, which means the cookie is only sent over HTTPS.

`clearTokenCookie(res)` clears the cookie by setting it to an empty string with `maxAge: 0` . This is used during logout.

The Auth Middleware

The auth middleware (`server/src/auth/auth.middleware.js`) is a function that sits between the incoming request and the route handler. When a route is "protected" (requires authentication), this middleware runs first.

It extracts the token from `req.cookies.token` . If no token exists, it immediately returns a 401 Unauthorized response. If a token is present, it verifies it using `jsonwebtoken.verify()` . If verification fails (expired, tampered, or invalid), it returns 401. If verification succeeds, it decodes the payload (which contains `{ id: userId }`) and sets `req.userId = decoded.id` . This makes the user's ID available to all downstream route handlers.

The Auth Routes

The auth routes (`server/src/auth/auth.routes.js`) handle four endpoints:

POST /api/auth/register accepts `{ name, email, password }`. It checks that all fields are present, checks that the email is not already in use (returns 400 if it is), creates the new User document, generates a JWT, sets the cookie, and returns the token and user information (id, name, email — never the password).

POST /api/auth/login accepts `{ email, password }`. It finds the user by email (explicitly requesting the password field with `.select("+password")`), compares the submitted password with the stored hash using `user.comparePassword(password)` , and if they match, generates a JWT and returns it with user info. If the email is not found or the password does not match, it returns 401.

GET /api/auth/me reads the token from the cookie, verifies it, looks up the user by the decoded ID, and returns their info. This endpoint is called when the React app first loads — if the user has a valid cookie from a previous session, this endpoint restores their login state without requiring them to log in again.

POST /api/auth/logout clears the token cookie and returns a success message. The browser discards the cookie, and the user is no longer authenticated.

How the Client Handles Auth

On the React side, the `AuthContext` (`client/src/context/AuthContext.jsx`) manages all authentication state. It provides a `user` object (or null), a `loading` boolean, and three functions: `register` , `login` , and `logout` .

When the `AuthProvider` first mounts, it calls `GET /api/auth/me` to check if there is an existing session. If the server returns a valid user, the `user` state is set. If not (no cookie, or expired token), `user` remains null. This is how "remember me" works — the cookie persists for 7 days, and as long as it exists and is valid, the user stays logged in across browser sessions.

The `ProtectedRoute` component in `App.jsx` checks whether `user` is set. If it is, the protected content renders. If not, the user is redirected to `/login` . This ensures that all main pages (Home, Create Idea, Idea Detail, Profile) are only accessible to authenticated users.

5. Ideas — The Core Domain Object

Ideas are the central entity in the application. Every other feature — voting, commenting, profiles — revolves around them.

The Idea Model

The Idea model (`server/src/ideas/idea.model.js`) defines five fields: `title` (a short string), `description` (a longer text), `status` (either "open" or "closed", defaulting to "open"), `author` (a reference to the User who created it), and two timestamps: `createdAt` and `updatedAt` .

The `updatedAt` field is maintained by a pre-save hook that runs before every save operation. When the status is changed, this hook sets `updatedAt` to the current time. The `status` field is constrained to exactly two values using an `enum` : "open" means the idea is accepting votes; "closed" means voting is locked.

Idea Routes

The idea routes (`server/src/ideas/idea.routes.js`) handle four operations:

GET /api/ideas lists all ideas. It supports three query parameters: `sort` (either "newest" or "popular"), `status` (either "open", "closed", or "all"), and `search` (a text string that is used as a case-insensitive regular expression against both the title and description fields). For each idea found, the route fetches all votes and computes the score using `computeScore()` . If sorting by popular, the ideas are then sorted in JavaScript by their score's good count in descending order.

GET /api/ideas/:id fetches a single idea by its MongoDB ObjectId. It populates the author's name (using Mongoose's `.populate()`), fetches all votes, computes the score, and returns the idea with its score attached.

POST /api/ideas (protected) creates a new idea. The `auth` middleware must pass first. The route extracts `title` and `description` from the request body and sets the `author` to `req.userId` (set by the auth middleware). It returns the newly created idea with a 201 status.

PATCH /api/ideas/:id (protected) updates an idea's status. It first checks that the authenticated user is the author of the idea. If they are not, it returns 403 Forbidden. If they are, it updates the status to whatever value is provided in the request body (must be "open" or "closed").

How Idea Routes Use Multiple Models

It is worth noting that the idea routes import not just the Idea model, but also the Vote model and the `computeScore` function. This is because the score is computed at read time: every time an idea is fetched, the route must also fetch all votes for that idea and compute the score. This coupling between the idea and vote domains is a deliberate design choice — scores are always fresh, at the cost of an extra database query per idea.

6. Voting — Binary Judgments with Change Capability

The voting system is designed around three principles: votes are binary (good or bad), each user can vote at most once per idea, and users can change their mind at any time.

The Vote Model

The Vote model (`server/src/votes/vote.model.js`) is the simplest model in the codebase. It has three fields: `idea` (reference to the Idea), `user` (reference to the User), and `value` (either "good" or "bad").

The most important feature of this model is the **compound unique index** on `{ idea: 1, user: 1 }`. This index enforces at the database level that there can be only one document with a given idea-user pair. If a user tries to create a second vote for the same idea, MongoDB will reject it. This is the ultimate safety net — even if the application code has a bug, the database itself prevents duplicate votes.

Vote Routes

The vote routes (`server/src/votes/vote.routes.js`) handle three operations:

POST /api/ideas/:id/votes (protected) casts or changes a vote. Before processing, it performs three checks: (1) the idea must exist, (2) the idea must be "open" (you cannot vote on a closed idea), and (3) the user must not be the author of the idea (you cannot vote on your own idea). If any check fails, it returns an appropriate error (404, 403, or 403). If all checks pass, it uses `findOneAndUpdate` with `upsert: true`. This single MongoDB operation either creates a new vote or updates an existing one — if the user already has a vote for this idea, it changes the value; if they don't, it creates a new one. After the vote is saved, it fetches all votes for the idea, computes the new score, and returns it.

GET /api/ideas/:id/votes/me (protected) returns the current user's vote on a specific idea, or null if they haven't voted. The React client uses this to determine which vote button to highlight when the page loads.

DELETE /api/ideas/:id/votes (protected) removes the current user's vote entirely. It uses `findOneAndDelete` to find and remove the vote document matching the idea and user. After deletion, it recomputes and returns the updated score.

The Upsert Pattern

The upsert pattern used in the POST handler is elegant. Instead of first checking if a vote exists, then deciding to insert or update, a single `findOneAndUpdate` with `upsert: true` handles both cases. If a document matching the filter (`{ idea, user }`) exists, it updates the `value` field. If no matching document exists, it creates one. This avoids race conditions and simplifies the code.

7. Comments — Discussion on Ideas

Comments allow users to discuss ideas. Unlike voting, which is restricted (one vote per user, no self-voting), commenting is open: anyone authenticated can comment on any idea, and authors can comment on their own ideas.

The Comment Model

The `Comment` model (`server/src/comments/comment.model.js`) has four fields: `text` (the comment content), `idea` (reference to the Idea), `author` (reference to the User), and `createdAt` and `updatedAt` timestamps. Like the `Idea` model, it has a pre-save hook that updates `updatedAt` whenever the comment is modified.

Comment Routes

The comment routes (`server/src/comments/comment.routes.js`) handle four operations:

GET `/api/ideas/:id/comments` lists all comments for a given idea, sorted by newest first. It uses Mongoose's `.populate("author", "name")` to include the author's name in each comment. This is a read-only endpoint — no authentication is required to view comments.

POST `/api/ideas/:id/comments` (protected) creates a new comment. It extracts the text from the request body, sets the idea and author from the URL parameter and authenticated user, saves the comment, populates the author's name, and returns it with a 201 status.

PATCH `/api/ideas/:ideaId/comments/:commentId` (protected) edits a comment. It checks that the authenticated user is the author of the comment (not the idea author — the comment author). If they are not the author, it returns 403. If they are, it updates the text and returns the updated comment.

DELETE `/api/ideas/:ideaId/comments/:commentId` (protected) deletes a comment. Here, the authorization check is different: the **idea author** (not the comment author) has the power to delete any comment on their idea. This is a design decision — the idea author moderates the discussion on their own idea. The route first fetches the idea to check authorship, then deletes the comment if authorized.

An Interesting Implementation Detail

In the DELETE handler, the `Idea` model is imported dynamically using `await import("../ideas/idea.model.js")` rather than being imported at the top of the file. This is a minor code organization choice — it avoids a static circular dependency between the comment routes and the idea model. In practice, it makes no functional difference, but it is a pattern you will see in some codebases to keep imports clean.

8. The Score Engine — How Votes Become a Percentage

The `computeScore()` function (`server/src/ideas/score.js`) is a pure function with no dependencies. It takes an array of vote objects and returns a score object with three numbers: the count of good votes, the count of bad votes, and a percentage.

How It Works

The function filters the votes array twice: once for votes where `value === "good"` and once for votes where `value === "bad"`. It counts the length of each filtered array. Then it calculates the percentage as `Math.round((good / total) * 100)`. If there are no votes at all (total is zero), it returns 0 for the percentage.

For example, if an idea has 7 good votes and 3 bad votes, the percentage is `Math.round(70) = 70`. If it has 0 good and 5 bad, the percentage is 0. If it has 2 good and 0 bad, the percentage is 100.

Where It Is Called

This function is called in three different route files:

- **idea.routes.js** — when listing all ideas (GET /) and when fetching a single idea (GET /:id)
- **vote.routes.js** — after casting, changing, or removing a vote, to return the updated score
- **user.routes.js** — when fetching a user's ideas (GET /users/:id/ideas)

Every time an idea is fetched, all of its votes must also be fetched so the score can be computed. This means a list of 10 ideas requires 10 additional vote queries plus 10 calls to `computeScore()`. This is a performance trade-off: the code is simple and scores are always accurate, but the database load increases with the number of ideas displayed.

Why Not Store the Score?

You might wonder why the score is not stored as a field on the Idea document, updated whenever a vote is cast. The reason is **consistency**. If the score were stored, you would need to update it in two places whenever a vote changes: the vote document and the idea document. If something goes wrong (a crash, a network error), the stored score could drift out of sync with the actual votes. By computing the score on every read, the system guarantees that the displayed score always matches the actual votes.

9. The React Client — How the Frontend Is Structured

The React client is a single-page application (SPA) built with Vite and React 19. When the user opens `http://localhost:3000`, the browser downloads a single HTML file and a JavaScript bundle. React then takes over, rendering the entire user interface and handling all navigation without full page reloads.

The Entry Point

The entry point is `client/src/main.jsx`. It imports the global CSS file (`index.css`), creates a React root, and renders the `App` component inside `StrictMode`. `StrictMode` is a React feature that helps catch common mistakes during development (it does nothing in production).

App.jsx — The Root Component

`client/src/App.jsx` is the most important client file. It does three things: it sets up routing, it renders the sidebar, and it defines the `ProtectedRoute` wrapper.

The entire app is wrapped in `BrowserRouter` (from `react-router-dom`), which enables client-side routing. Inside that is `AuthProvider` (from `AuthContext`), which provides authentication state to all child components. Inside that is `AppLayout`, which contains the actual layout.

The `AppLayout` component conditionally renders the `Sidebar` (only if the user is logged in) and a `Routes` element that maps URL paths to page components. If the user is not logged in, the sidebar is hidden, and only the login/register routes are accessible.

The `Sidebar` component renders a fixed left column with navigation links (Home, New Idea, Profile) and a logout button. Each link uses a `lucide-react` icon. The sidebar also shows the current user's name and email at the bottom. The active link is highlighted using the current URL path (checked via `useLocation()`).

The `ProtectedRoute` component receives children as a prop. It checks the auth context: if loading is true, it shows a loading indicator; if the user exists, it renders the children; if not, it redirects to `/login` using `Navigate`.

10. Page by Page — What Each React Component Does

Home.jsx — The Idea Feed

The Home page (`client/src/pages/Home.jsx`) is the main landing page after login. It displays a list of all ideas in the system, with filtering, sorting, and search capabilities.

The component maintains three pieces of filter state: `sort` (newest or popular), `status` (all, open, or closed), and `search` (a text string). Whenever any of these change, it fetches fresh data from `GET /api/ideas` with the appropriate query parameters. The search is debounced with a 300ms delay — as the user types, the component waits 300ms after the last keystroke before making the API call, preventing excessive requests.

Each idea is rendered as a card showing the author's name, the title, a truncated description (limited to 2 lines), the status badge, and the vote counts (good/bad) with icons. Clicking on a card navigates to the idea detail page.

IdeaDetail.jsx — The Most Complex Component

The Idea Detail page (`client/src/pages/IdeaDetail.jsx`) is the most feature-rich component in the application. It handles viewing a single idea, voting on it, and managing comments. It makes 9 different API calls.

When the page loads, it fetches three things in parallel: the idea itself, its comments, and the current user's vote. It then displays the idea's title, description, author, status, and score. Below the idea, it shows vote buttons (good/bad) if the user is allowed to vote (not the author, and the idea is open). The currently active vote is highlighted.

The voting system allows toggling: if the user clicks the button that matches their current vote, the vote is removed. If they click the other button, their vote changes. The UI updates immediately after each action.

Below the voting section is the comments area. Users can type a comment in a textarea and submit it. Existing comments are listed in reverse chronological order. Each comment shows the author's name, the date, and the text. If the current user is the comment author, an Edit button appears. If the current user is the idea author, a Delete button appears on every comment. The idea author can also close or reopen the idea using a button near the status badge.

CreateIdea.jsx — The Idea Submission Form

The Create Idea page (`client/src/pages/CreateIdea.jsx`) is a simple form with two fields: title and description. When submitted, it sends a POST request to `/api/ideas` and, on success, navigates to the newly created idea's detail page. A back arrow button allows the user to return to the previous page.

Profile.jsx — User Profile

The Profile page (`client/src/pages/Profile.jsx`) shows a user's profile information and all their ideas. It takes a user ID from the URL parameter, fetches that user's ideas via `GET /api/users/:id/ideas`, and displays them as cards (similar to the Home feed but without the author line, since all ideas belong to the same user).

Login.jsx and Register.jsx — Authentication Forms

These two pages are nearly identical in structure. Both display a centered form with the `Lightbulb` icon at the top as branding. The Login page has email and password fields; the Register page adds a name field. Both call functions from the `AuthContext` (`login()` or `register()`), which in turn make API calls to the server. On success, the user is redirected to the home page. On failure, an error message is displayed.

11. Styling — How index.css Controls the Entire UI

All styling for the application lives in a single file: `client/src/index.css`. The file `App.css` exists but is empty — all styles were consolidated into `index.css` during the UI redesign.

The Design System

The CSS defines a set of CSS custom properties (variables) on the `:root` element that create a consistent light theme:

- `--bg` (`#fafafa`) — the page background color
- `--fg` (`#0f1419`) — the primary text color
- `--muted` (`#536471`) — secondary, less prominent text
- `--border` (`#eff3f4`) — borders around cards, inputs, and sections
- `--input-bg` (`#eff3f4`) — the background of input fields and hover states
- `--accent` (`#0f1419`) — the dark color used for buttons and active states

The overall visual style is clean and minimal: a light gray background, white cards, dark text, and subtle borders. Buttons are pill-shaped (using `border-radius: 9999px`). The layout uses flexbox throughout, with a sticky sidebar on the left and a centered main content column (max-width 600px) in the middle.

Key Layout Classes

`.app` is a flex container that holds the sidebar and main content side by side. `.sidebar` is 260px wide, sticky to the top, and fills the viewport height. `.main-content` is centered with `margin: 0 auto` and capped at 600px width, with borders on both sides.

`.main-header` is a sticky header that stays at the top of the content area as you scroll. It uses a semi-transparent background with a blur effect (`backdrop-filter: blur(12px)`) to create a frosted glass appearance.

Idea cards (class `.idea-card`) are simple rows with a bottom border. They show the author, title, excerpt, and vote counts. The hover state changes the background to `var(--input-bg)`.

Responsive Design

A media query at `max-width: 768px` hides the sidebar entirely and makes the main content full-width. The filter bar on the home page stacks vertically on narrow screens.

12. The Connection — How Frontend Talks to Backend

Every interaction between the React client and the Express server follows the same pattern: the client makes an HTTP request using Axios, the server processes it through its middleware and route handlers, and the server sends back a JSON response. The client then updates its React state based on that response.

The Proxy Mechanism

The Vite dev server is configured (in `vite.config.js`) to proxy all requests matching `/api` to `http://localhost:5000`. This means when the React code calls `axios.get("/api/ideas")`, the request goes to `localhost:3000/api/ideas`, which Vite intercepts and forwards to `localhost:5000/api/ideas`. The response comes back through Vite to the browser. This is transparent to the React code — it just sees `/api/*` as if everything is on the same server.

Cookie Flow

When the user logs in, the server sets an `httpOnly` cookie in the response. Axios is configured (in `AuthContext`) with `{ withCredentials: true }`, which tells the browser to include cookies on all subsequent requests. So when the user navigates to the home page and the React app calls `GET /api/ideas`, the browser automatically includes the JWT cookie. The Express server's `cookie-parser` middleware extracts it, and the `auth` middleware (on protected routes) verifies it.

This is fundamentally different from the more common pattern of storing JWTs in `localStorage` and sending them in an `Authorization` header. The cookie-based approach is more secure against XSS attacks because the JavaScript running in the browser cannot read the cookie value.

The Complete Request Lifecycle

Here is the complete lifecycle of a vote request, from button click to screen update:

1. The user clicks the "Good" vote button in `IdeaDetail.jsx`
2. The `vote("good")` function is called
3. Axios sends `POST /api/ideas/:id/votes` with `{ value: "good" }` in the body and the JWT cookie in the Cookie header
4. Vite intercepts the request and proxies it to port 5000
5. Express receives the request and passes it through CORS, JSON parser, and cookie parser middleware
6. The vote route handler runs the `auth` middleware, which extracts and verifies the JWT, setting `req.userId`
7. The handler validates that the idea exists, is open, and the user is not the author
8. The handler uses `findOneAndUpdate` with `upsert: true` to create or update the vote
9. The handler fetches all votes for the idea and calls `computeScore()`
10. The handler returns `{ score: { good: 8, bad: 2, percentage: 80 } }`
11. Axios resolves the promise in the React component
12. The component calls `fetchIdea()` and `fetchUserVote()` to refresh all data
13. React re-renders the `IdeaDetail` component with the updated score and vote state
14. The user sees the new score on their screen

13. Testing — How the Test Suite Works

The server uses Node.js's built-in test runner (`node:test`) along with `supertest` , a library for testing HTTP servers. There are no client-side tests.

Test Organization

There are 6 test files containing 34 tests total. Each test file focuses on one domain:

- **health.test.js** (1 test) — confirms the health endpoint returns OK
- **auth.test.js** (7 tests) — tests registration, login, me endpoint, and logout
- **idea.test.js** (7 tests) — tests idea CRUD and authorization
- **score.test.js** (5 tests) — unit tests for the `computeScore` function
- **vote.test.js** (6 tests) — tests voting, self-vote prevention, and closed-idea voting
- **comment.test.js** (6 tests) — tests comment creation, editing, and authorization

Test Isolation

Each test file connects to its own MongoDB database (e.g., `idea-vault-test-auth` , `idea-vault-test-ideas`). This prevents tests from interfering with each other when they run in parallel (which Node.js does by default). Before each test suite, the relevant collections are cleared. After the suite, the database connection is closed.

The test databases are separate from the development database (`idea-vault`), so running tests never destroys your development data.

How to Run Tests

```
# Run all tests from the project root
npm test

# Run all tests from the server directory
npm test --prefix server

# Expected output: 34 tests passing
```

Important: MongoDB must be running before you can execute the tests, since the test files connect to MongoDB to create users, ideas, and other test data.

The Test Pattern

Most integration tests follow the same pattern: the `before` hook connects to MongoDB and creates the necessary data (users, ideas). Each `it` block uses `supertest` to make HTTP requests against the Express app and asserts on the response status and body. The `after` hook cleans up and closes the connection. This gives each test a clean slate.

The `score.test.js` file is different — it is a pure unit test. It imports the `computeScore` function directly and tests it with various vote arrays, without touching the database or making HTTP requests.